



Capítulo 5 - Leitura de vários sensores

Ler cada sensor, reconhecer mudanças e manter o programa disponível para as outras tarefas.

Introdução	No capítulo anterior, organizamos o comando de vários desvios. Agora vamos reunir as entradas de vários sensores e observar seus estados separadamente.
Objetivo	Ler três entradas digitais com uma ESP32, identificar o sensor acionado e apresentar mensagens quando houver mudança de estado.
Meta de aprendizagem	Ao final, você deverá conseguir mapear sensores e GPIOs, interpretar uma entrada ativa em nível baixo e aplicar uma verificação simples de estabilidade.

1. O que um sensor informa?

Um sensor fornece uma informação ao programa. Neste capítulo, utilizaremos entradas digitais: o sinal será lido como HIGH ou LOW.

Um sensor Hall detecta um campo magnético. No ferromodelismo, um ímã colocado sob uma locomotiva ou veículo pode acioná-lo quando passa pelo ponto de instalação.

Um sensor Hall acionado informa a presença de campo magnético naquele ponto. Isso não confirma, sozinho, que todo um trecho está ocupado ou livre. A identificação da posição dos trens será desenvolvida no próximo capítulo.

Os sensores deste exercício representam pontos de passagem na ferrovia. Eles não são sensores de posição das agulhas dos desvios.

2. Identificação e GPIO de cada sensor

Usaremos S10, S11 e S12 como exemplos de sensores ligados à gaveta GA3. O mapeamento abaixo é didático, para uma montagem de bancada, e não altera as conexões definitivas da gaveta GA3.

Sensor	Índice no programa	GPIO de exemplo
S10	0	32
S11	1	33
S12	2	27

Esses pinos são exemplos para uma ESP32 clássica baseada no ESP32-WROOM. Verifique a disponibilidade dos GPIOs na placa utilizada. Cada sensor precisa de seu próprio fio de sinal.



3. Entrada ativa em nível baixo

No sensor Hall A3144 de saída coletor aberto, o transistor de saída conduz quando o campo magnético adequado aciona o sensor. Com a ligação prevista neste capítulo, o sinal fica próximo de GND e a ESP32 lê LOW.

Quando a saída deixa de conduzir, um resistor de pull-up mantém o sinal em nível alto. Pull-up significa uma ligação resistiva que leva o sinal ao nível alto quando a saída não o leva ao GND.

Leitura	Interpretação neste exercício
LOW	Sensor acionado.
HIGH	Sensor não acionado.

```
bool active = digitalRead(sensorPins[index]) == LOW;
```

A expressão compara a leitura com LOW. O resultado será true se o sensor estiver acionado e false se não estiver.

4. Ligações para o teste de bancada

Antes de utilizar os sensores Hall, podemos simular as três entradas com botões ou contatos. Cada entrada terá um resistor externo de 4,7 kΩ entre o sinal e o pino 3V3 da ESP32.

Elemento	Ligação
Resistor de S10	GPIO 32 para 3V3, por meio de 4,7 kΩ.
Resistor de S11	GPIO 33 para 3V3, por meio de 4,7 kΩ.
Resistor de S12	GPIO 27 para 3V3, por meio de 4,7 kΩ.
Contato de cada entrada	Entre seu GPIO e GND. Fechar o contato simula o acionamento.

Para o A3144 discreto, alimente o sensor com 5 V, ligue seu GND ao GND da ESP32 e conecte a saída ao GPIO correspondente, mantendo o resistor de pull-up ligado a 3V3.

Assim, o sensor recebe 5 V de alimentação, mas o nível alto do sinal é definido pelo pull-up de 3,3 V. Não conecte o sinal nem o resistor de pull-up a 5 V na entrada da ESP32.

Essa ligação vale para o A3144 discreto de saída coletor aberto. Módulos prontos podem ter resistores ou circuitos internos que elevam a saída a 5 V. Confira o circuito do módulo antes de utilizá-lo. Verifique também a ordem dos terminais do sensor na documentação da peça adquirida.

Padrão dos cabos dos sensores: preto para GND, vermelho para +5 V e verde para o sinal. O resistor de pull-up vai do sinal ao 3V3, não ao fio vermelho.



Monte com a alimentação desligada. O primeiro teste pode ser feito somente com botões, resistores e a ESP32 alimentada por USB.

5. Mostrar o estado sem repetir mensagens

Se uma mensagem for escrita a cada passagem por `loop()`, o Monitor Serial ficará cheio de linhas iguais. Vamos guardar o estado de cada sensor e mostrar uma mensagem somente quando ele mudar.

Na inicialização, o programa também mostrará o estado de todas as entradas. Depois, informará apenas as mudanças reconhecidas.

```
S10 - ACTIVE  
S10 - INACTIVE
```

A primeira mensagem significa que S10 foi reconhecido como acionado. A segunda indica que voltou ao estado não acionado.

6. Verificação de estabilidade

Um botão pode apresentar pequenas oscilações no contato. Uma entrada também pode sofrer interferência elétrica. Neste exercício, uma nova leitura precisa permanecer igual por 30 milissegundos antes de ser aceita.

Guardaremos três informações para cada sensor: o estado aceito, a leitura candidata e o instante em que essa leitura candidata mudou.

Quando a leitura muda, o programa reinicia a contagem de estabilidade daquela entrada. Se o valor permanecer igual durante o intervalo, ele passa a ser o estado aceito.

O filtro acrescenta um atraso e pode ignorar acionamentos curtos. Os 30 ms são um valor didático para a bancada. Na ferrovia, o intervalo precisa ser testado com a duração da passagem do ímã e com a maior velocidade prevista, sem perder eventos.

Utilizaremos `millis()` para controlar o intervalo. Não haverá `delay()` dentro do acompanhamento contínuo das entradas.

7. Programa completo

Crie um programa chamado `intermediate_05_sensors`. Não é necessária uma biblioteca adicional. Configure as ligações do item 4 antes do teste físico.



```
const int sensorCount = 3;
const int sensorIds[sensorCount] = {10, 11, 12};
const int sensorPins[sensorCount] = {32, 33, 27};
const unsigned long stableInterval = 30;
bool stableStates[sensorCount];
bool candidateStates[sensorCount];
unsigned long changedAt[sensorCount];

void showSensorState(int index) {
    Serial.print("S");
    Serial.print(sensorIds[index]);
    Serial.println(stableStates[index] ? " - ACTIVE" : " - INACTIVE");
}

void readSensors() {
    for (int index = 0; index < sensorCount; index++) {
        bool active = digitalRead(sensorPins[index]) == LOW;
        unsigned long now = millis();
        if (active != candidateStates[index]) {
            candidateStates[index] = active;
            changedAt[index] = now;
        } else if (active != stableStates[index] &&
            now - changedAt[index] >= stableInterval) {
            stableStates[index] = active;
            showSensorState(index);
        }
    }
}
```

Inicialização e leitura contínua

```
void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("EFMN 74 - Sensor monitoring");
    for (int index = 0; index < sensorCount; index++) {
        pinMode(sensorPins[index], INPUT);
        bool active = digitalRead(sensorPins[index]) == LOW;
        stableStates[index] = active;
        candidateStates[index] = active;
        changedAt[index] = millis();
        showSensorState(index);
    }
}

void loop() {
    readSensors();
}
```

8. Entendendo a leitura conjunta

sensorIds guarda as identificações S10, S11 e S12. sensorPins guarda os GPIOs, na mesma ordem. As demais listas mantêm o estado e o tempo de cada entrada separadamente.

pinMode(..., INPUT) configura a entrada. Como o circuito utiliza um resistor externo de pull-up, não dependemos do pull-up interno da ESP32.

O for faz uma leitura curta de cada sensor. Os três sensores são verificados em sequência a cada passagem por loop(); não são lidos exatamente no mesmo instante.



A expressão `now - changedAt[index]`, com valores `unsigned long`, calcula o tempo transcorrido. Essa forma de comparação continua funcionando quando o contador de `millis()` reinicia após atingir seu limite.

A leitura inicial é apresentada imediatamente e usada como ponto de partida. O filtro de estabilidade passa a atuar nas mudanças posteriores.

Exercício prático

Meta: reconhecer cada entrada e verificar mudanças individuais e combinadas.

1. Monte as três entradas com resistores de 4,7 k Ω e contatos para GND, conforme o item 4.
2. Confira as ligações com a alimentação desligada.
3. Conecte a ESP32 de bancada por USB, selecione a placa e a porta e compile o programa.
4. Envie o programa e abra o Monitor Serial em 115200 baud.
5. Com os contatos abertos, pressione EN ou RESET.
6. Confira os três estados iniciais não acionados.
7. Feche o contato de S10 por aproximadamente um segundo e depois abra.
8. Repita o teste com S11 e S12, um de cada vez.
9. Mantenha dois contatos fechados ao mesmo tempo e confira as duas identificações.
10. Solte apenas um deles e confira que somente seu estado muda.

Resultado inicial esperado, com todos os contatos abertos:

```
EFMN 74 - Sensor monitoring  
S10 - INACTIVE  
S11 - INACTIVE  
S12 - INACTIVE
```

Ao fechar e abrir o contato de S11:

```
S11 - ACTIVE  
S11 - INACTIVE
```

Ao manter um contato fechado, a mensagem `ACTIVE` deve aparecer apenas uma vez após o reconhecimento. Não deve continuar se repetindo enquanto o estado permanecer igual.

Teste com os sensores Hall

1. Desligue a alimentação e substitua um dos contatos pelo sensor Hall corretamente ligado.
2. Confira a alimentação de 5 V, o GND comum e o pull-up para 3V3.
3. Antes de ligar o sinal ao GPIO, confira com o multímetro o nível alto previsto de aproximadamente 3,3 V.
4. Aproxime o ímã e confira a mudança para `ACTIVE`. Se não houver acionamento, experimente o outro polo e confira a posição da face sensível.



5. Afaste o ímã e confira o retorno para INACTIVE.
6. Repita separadamente com os demais sensores.
7. Teste a passagem do ímã na velocidade prevista e registre se todas as passagens foram reconhecidas.

9. O que verificar quando a leitura estiver errada

Comportamento	Verificação
Entrada oscila sem acionamento.	Confira pull-up, GND comum, contatos, fios e interferência.
Entrada permanece sempre ACTIVE.	Confira curto do sinal para GND, ímã próximo e a ligação da saída.
Entrada permanece sempre INACTIVE.	Confira alimentação do sensor, GPIO, terminais, posição do ímã e continuidade do sinal.
Aparece a identificação de outro sensor.	Confira a ordem das listas e o mapeamento físico dos fios.
Passagens rápidas não são reconhecidas.	Confira a duração do pulso, o intervalo de estabilidade e o tempo gasto nas outras tarefas.

Uma entrada em HIGH também pode ocorrer quando o fio do sensor está desconectado, pois o resistor de pull-up mantém o nível alto. Este circuito não identifica sozinho uma falha de cabo.

10. Registro dos testes

Para cada sensor, registre a identificação, o GPIO, a tensão do sinal em repouso, a resposta ao ímã e o resultado dos testes de passagem.

Esses registros ajudam a distinguir problemas de montagem, ajuste do ímã e interpretação do programa. A identificação da posição dos trens será construída a partir dessas informações, sem tratar uma leitura isolada como confirmação de ocupação de todo o trecho.



Resumo do capítulo

- Cada sensor precisa de uma entrada de sinal própria.
- As listas mantêm os GPIOs e os estados na mesma ordem.
- Neste circuito, LOW significa sensor acionado.
- O A3144 recebe 5 V, enquanto o sinal para a ESP32 utiliza pull-up para 3V3.
- Mensagens de mudança evitam repetir informações iguais.
- O filtro de estabilidade precisa respeitar a duração dos acionamentos.
- Verificações curtas permitem acompanhar várias entradas continuamente.
- Sensor não acionado não significa, sozinho, trecho livre.

Próximo capítulo: Identificação da posição dos trens.

Referências técnicas

Allegro Microsystems: folha de dados A3141, A3142, A3143 e A3144.

<https://www.allegromicro.com/~media/Files/Datasheets/A3141-2-3-4-Datasheet.ashx>

Espressif: folha de dados ESP32, características elétricas dos GPIOs.

https://documentation.espressif.com/esp32_datasheet_en.html